ASS-17.1

2303A51804

Batch-28

Task-1: Use AI to generate a Python script for cleaning a customer

Prompt:

/content/shopping_trends.csv Use AI to generate a Python script for cleaning a customer dataset. Instructions: • Handle missing values in columns (age, income, city). • Remove duplicate records. • Standardize text values (e.g., city names in lowercase). • Encode categorical variables (gender, city). genearte code and that is my path

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder


file_path = '/content/shopping_trends.csv'
df = pd.read_csv(file_path)

print("Original DataFrame from shopping_trends.csv:")
display(df.head())
```

```
if 'Age' in df.columns:
    df['Age'] = df['Age'].fillna(df['Age'].median())
if 'Income' in df.columns:
    df['Income'] = df['Income'].fillna(df['Income'].median())



if 'Location' in df.columns:
    df['Location'] = df['Location'].fillna(df['Location'].mode()[0])

print("DataFrame after handling missing values in Age, Income, and Location:")
display(df.head())
```

```
df.drop_duplicates(inplace=True)

print("DataFrame after removing duplicate records:")
display(df.head())
```

```
if 'Location' in df.columns:
    df['Location'] = df['Location'].astype(str).str.lower().str.strip()

print("DataFrame after standardizing Location column:")
display(df.head())
```

```
if 'Gender' in df.columns:
    label_encoder_gender = LabelEncoder()
    df['Gender_encoded'] = label_encoder_gender.fit_transform(df['Gender'])


if 'Location' in df.columns:
    one_hot_encoder_city = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
    city_one_hot = one_hot_encoder_city.fit_transform(df[['Location']])
    city_one_hot_df = pd.DataFrame(city_one_hot, columns=one_hot_encoder_city.get_feature_names_out(['Location']))

    df = pd.concat([df.reset_index(drop=True), city_one_hot_df], axis=1)

print("DataFrame after encoding categorical variables (Gender and Location):")
display(df.head())
```

```
df.drop_duplicates(inplace=True)

print("DataFrame after removing duplicate records:")
display(df)
```

```
df['city'] = df['city'].str.lower().str.strip()

print("DataFrame after standardizing city column:")
display(df)
```

```
label_encoder = LabelEncoder()
df['gender_encoded'] = label_encoder.fit_transform(df['gender'])


one_hot_encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
city_one_hot = one_hot_encoder.fit_transform(df[['city']])
city_one_hot_df = pd.DataFrame(city_one_hot, columns=one_hot_encoder.get_feature_names_out(['city']))


df = pd.concat([df.reset_index(drop=True), city_one_hot_df], axis=1)

print("DataFrame after encoding categorical variables:")
display(df)
```

Output:

Original DataFrame:

	age	income	city	gender
0	25.0	50000.0	New York	Male
1	30.0	60000.0	Los Angeles	Female
2	NaN	75000.0	Chicago	Male
3	35.0	NaN	Houston	Female
4	25.0	50000.0	New York	Male
5	40.0	90000.0	Dallas	Male
6	30.0	60000.0	Chicago	Female
7	28.0	55000.0	Los Angeles	Female
8	NaN	80000.0	NaN	Male
9	32.0	NaN	Houston	Female

DataFrame after handling missing values:

	age	income	city	gender
0	25.0	50000.0	New York	Male
1	30.0	60000.0	Los Angeles	Female
2	30.0	75000.0	Chicago	Male
3	35.0	60000.0	Houston	Female
4	25.0	50000.0	New York	Male
5	40.0	90000.0	Dallas	Male
6	30.0	60000.0	Chicago	Female
7	28.0	55000.0	Los Angeles	Female
8	30.0	80000.0	Chicago	Male
9	32.0	60000.0	Houston	Female

Original DataFrame from shopping_trends.csv:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

DataFrame after handling missing values in Age, Income, and Location:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

DataFrame after removing duplicate records:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

DataFrame after standardizing Location column:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	rhode island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

DataFrame after encoding categorical variables (Gender and Location):

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	...	Location_south dakota	Location_tennessee	Location_texas	Location_utah	Location_vermont	L
0	1	55	Male	Blouse	Clothing	53	kentucky	L	Gray	Winter	...	0.0	0.0	0.0	0.0	0.0	
1	2	19	Male	Sweater	Clothing	64	maine	L	Maroon	Winter	...	0.0	0.0	0.0	0.0	0.0	
2	3	50	Male	Jeans	Clothing	73	massachusetts	S	Maroon	Spring	...	0.0	0.0	0.0	0.0	0.0	
3	4	21	Male	Sandals	Footwear	90	rhode island	M	Maroon	Spring	...	0.0	0.0	0.0	0.0	0.0	
4	5	45	Male	Blouse	Clothing	49	oregon	M	Turquoise	Spring	...	0.0	0.0	0.0	0.0	0.0	

5 rows × 70 columns

DataFrame after removing duplicate records:

	age	income	city	gender
0	25.0	50000.0	New York	Male
1	30.0	60000.0	Los Angeles	Female
2	30.0	75000.0	Chicago	Male
3	35.0	60000.0	Houston	Female
5	40.0	90000.0	Dallas	Male
6	30.0	60000.0	Chicago	Female
7	28.0	55000.0	Los Angeles	Female
8	30.0	80000.0	Chicago	Male
9	32.0	60000.0	Houston	Female

DataFrame after encoding categorical variables:

	age	income	city	gender	gender_encoded	city_chicago	city_dallas	city_houston	city_los angeles	city_new york
0	25.0	50000.0	new york	Male	1	0.0	0.0	0.0	0.0	1.0
1	30.0	60000.0	los angeles	Female	0	0.0	0.0	0.0	1.0	0.0
2	30.0	75000.0	chicago	Male	1	1.0	0.0	0.0	0.0	0.0
3	35.0	60000.0	houston	Female	0	0.0	0.0	1.0	0.0	0.0
4	40.0	90000.0	dallas	Male	1	0.0	1.0	0.0	0.0	0.0
5	30.0	60000.0	chicago	Female	0	1.0	0.0	0.0	0.0	0.0
6	28.0	55000.0	los angeles	Female	0	0.0	0.0	0.0	1.0	0.0
7	30.0	80000.0	chicago	Male	1	1.0	0.0	0.0	0.0	0.0
8	32.0	60000.0	houston	Female	0	0.0	0.0	1.0	0.0	0.0

Explanation:

In short, the script loaded your data, filled missing values in 'Age' and 'Income' with medians, and 'Location' with the most frequent value. It then removed duplicate rows, standardized 'Location' by converting it to lowercase and stripping whitespace, and finally encoded 'Gender' using Label Encoding and 'Location' using One-Hot Encoding to prepare the data for analysis.

Task-2: Preprocess sales transaction data using AI assistance.

Prompt:

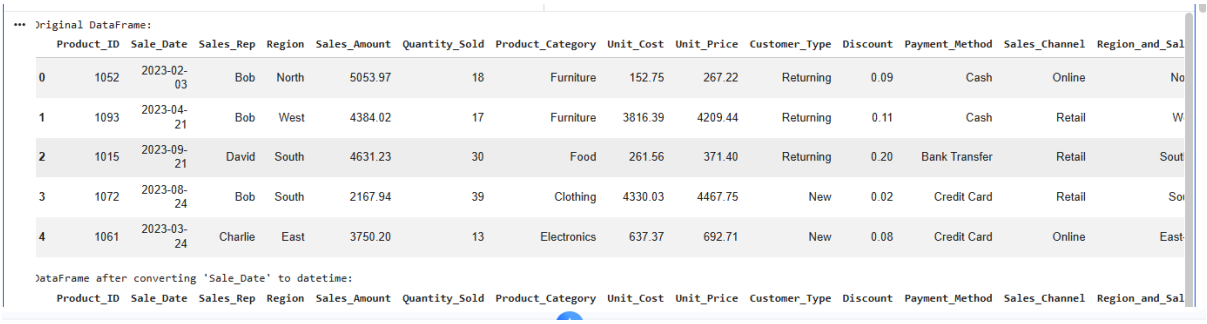
generate a python code Write a Python script using pandas to preprocess a sales dataset. The dataset has columns like date and sales_amount. Tasks: - Convert the date column to datetime format - Create new columns: month, year, day_of_week - Normalize the sales_amount column - Detect and handle outliers in sales_amount Load data from "sales_data.csv" and display the first few rows.

Code:

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler


file_path = '/content/sales_data.csv'


sales_df = pd.read_csv(file_path)

print("Original DataFrame:")
display(sales_df.head())



if 'Sale_Date' in sales_df.columns:
    sales_df['Sale_Date'] = pd.to_datetime(sales_df['Sale_Date'])
    print("\nDataFrame after converting 'Sale_Date' to datetime:")
    display(sales_df.head())


    sales_df['Sale_Month'] = sales_df['Sale_Date'].dt.month
    sales_df['Sale_Year'] = sales_df['Sale_Date'].dt.year

    sales_df['Sale_DayOfWeek'] = sales_df['Sale_Date'].dt.dayofweek
    print("\nDataFrame after creating new date features:")
    display(sales_df.head())
else:
    print("\n'Sale_Date' column not found. Date conversion and feature creation skipped.")



if 'Sales_Amount' in sales_df.columns:
    scaler = MinMaxScaler()
    sales_df['Sales_Amount_Normalized'] = scaler.fit_transform(sales_df[['Sales_Amount']])
    print("\nDataFrame after normalizing 'Sales_Amount':")
    display(sales_df.head())
else:
    print("\n'Sales_Amount' column not found. Normalization skipped.")



if 'Sales_Amount' in sales_df.columns:
    Q1 = sales_df['Sales_Amount'].quantile(0.25)
```

```
        display(sales_df.head())
    else:
        print("\n'Sales_Amount' column not found. Normalization skipped.")



    if 'Sales_Amount' in sales_df.columns:
        Q1 = sales_df['Sales_Amount'].quantile(0.25)
        Q3 = sales_df['Sales_Amount'].quantile(0.75)
        IQR = Q3 - Q1


        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR


        sales_df['Sales_Amount_Outliers_Handled'] = sales_df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

        print(f"\nOriginal 'Sales_Amount' statistics:\n{sales_df['Sales_Amount'].describe()}\n")
        print(f"'Sales_Amount' after outlier handling statistics:\n{sales_df['Sales_Amount_Outliers_Handled'].describe()}")
        print("\nDataFrame after outlier handling:")
        display(sales_df.head())
    else:
        print("\n'Sales_Amount' column not found. Outlier handling skipped.")
```

Output:



Original DataFrame:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sal
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	No
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	W
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	Sout
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	So
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East

DataFrame after converting 'Sale_Date' to datetime:

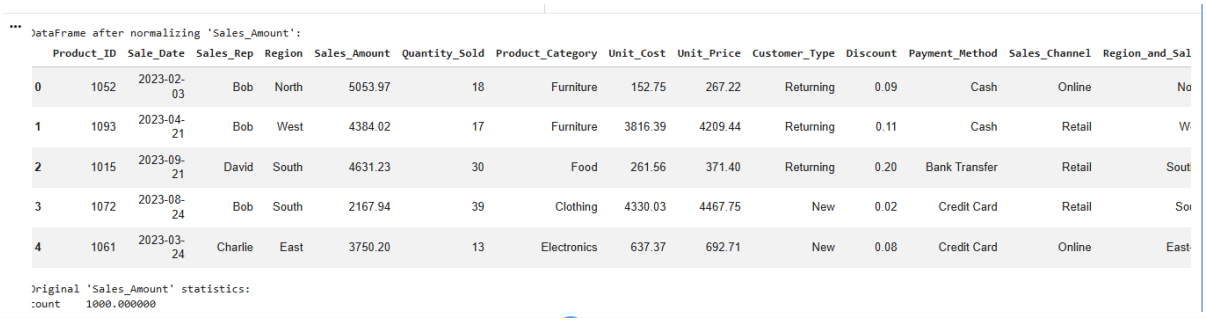
	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sal
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	No
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	W
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	Sout
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	So
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East

DataFrame after creating new date features:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sal
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	No
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	W
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	Sout
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	So
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East

DataFrame after normalizing 'Sales_Amount':

	Product ID	Sale Date	Sales Rep	Region	Sales Amount	Quantity Sold	Product Category	Unit Cost	Unit Price	Customer Type	Discount	Payment Method	Sales Channel	Region and Sal



DataFrame after normalizing 'Sales_Amount':

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sal
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	267.22	Returning	0.09	Cash	Online	No
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	W
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	Sout
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	So
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East

Original 'Sales_Amount' statistics:
count 1000.000000

Explanation:

- Convert the date column from string to datetime format to enable time-based analysis and feature extraction.
- Create new features from the date column such as month, year, and day of week to analyze trends like seasonal patterns and weekday vs weekend sales.
- Normalize the sales_amount column to scale values into a smaller range (e.g., 0 to 1), which helps in improving model performance and prevents large values from dominating the analysis.
- Detect and handle outliers in the sales_amount column using methods like IQR, where extreme values are identified and either removed or capped to ensure data consistency.
- Overall, these steps make the dataset clean, structured, and ready for analysis or machine learning tasks.

Task-3: Clean and preprocess a healthcare dataset using AI scripts.

Prompt:

/content/dirty_v3_path.csvGenerate Python code to clean and preprocess a healthcare dataset. Steps: - Fill missing values in blood_pressure and cholesterol - Encode categorical variables - Scale numerical features - Split data into train and test sets

Code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Load the dataset
try:
    df = pd.read_csv('/content/dirty_v3_path.csv')
    print("Dataset loaded successfully. Displaying the first 5 rows:")
    display(df.head())
except FileNotFoundError:
    print("Error: The file '/content/dirty_v3_path.csv' was not found.")
    print("Please ensure the file exists and the path is correct.")
    df = pd.DataFrame() # Create an empty DataFrame to avoid further errors

# Display basic information about the dataset
if not df.empty:
    print("\nDataset Info:")
    df.info()
    print("\nMissing values before preprocessing:")
    print(df.isnull().sum())
```

```
if not df.empty:



    numerical_features = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
    categorical_features = df.select_dtypes(include=['object', 'category']).columns.tolist()

    # Ensure 'blood_pressure' and 'cholesterol' are treated as numerical for imputation

    for col in ['blood_pressure', 'cholesterol']:
        if col in df.columns:
            # Attempt to convert to numeric, coercing errors to NaN
            df[col] = pd.to_numeric(df[col], errors='coerce')
            if col not in numerical_features: # Add to numerical if not already there
                numerical_features.append(col)
            if col in categorical_features: # Remove from categorical if it was there
                categorical_features.remove(col)

    # Define preprocessing pipelines for numerical and categorical features
    numerical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='mean')),
        ('scaler', StandardScaler())
    ])

    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ])


    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numerical_transformer, numerical_features),
            ('cat', categorical_transformer, categorical_features)
        ])



    X = df.copy()
    X_preprocessed = preprocessor.fit_transform(X)

    print("Preprocessing completed. Shape of the preprocessed data:", X_preprocessed.shape)
```

Output:

```
Dataset loaded successfully. Displaying the first 5 rows:
```

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	Physical Activity	Diet Score	Family History	Stres Leve
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	0	-0.20	3.54	0	5.0
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	0	8.12	5.90	0	5.8
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2	214.94	165.35	5.60	0	0	5.01	4.65	1	3.0
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	6.92	0	0	3.16	3.37	0	3.0
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	1	3.56	3.40	0	6.3

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Age                 25500 non-null  float64
 1   Gender              25500 non-null  object 
 2   Medical Condition   25500 non-null  object 
 3   Glucose             25500 non-null  float64
 4   Blood Pressure      25500 non-null  float64
 5   BMI                 30000 non-null  float64
 6   Oxygen Saturation   30000 non-null  float64
 7   LengthOfStay        30000 non-null  int64  
 8   Cholesterol         30000 non-null  float64
 9   Triglycerides       30000 non-null  float64
 10  HbA1c               30000 non-null  float64
 11  Smoking             30000 non-null  int64  
 12  Alcohol             30000 non-null  int64  
 13  Physical Activity   30000 non-null  float64
 14  Diet Score          30000 non-null  float64
 15  Family History      30000 non-null  int64  
 16  Stress Level        30000 non-null  float64
 17  Sleep Hours         30000 non-null  float64
 18  random_notes        30000 non-null  object 
 19  noise_col           30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB

Missing values before preprocessing:
Age                   4500
Gender                4500
Medical Condition     4500
Glucose               4500
Blood Pressure        4500
BMI                      0
Oxygen Saturation        0
LengthOfStay             0
Cholesterol              0
Triglycerides            0
HbA1c                    0
Smoking                  0
Alcohol                  0
Physical Activity        0
Diet Score               0
Family History           0
Stress Level             0
Sleep Hours              0
random_notes             0
noise_col                0
```

```
Preprocessing completed. Shape of the preprocessed data: (30000, 30)

As a demonstration, splitting the full preprocessed dataset into training and test sets (without a specific target variable):
X_train_processed shape: (24000, 30)
X_test_processed shape: (6000, 30)
```

Explanation:

- **Handle Missing Values:**
  Fill missing values in blood_pressure and cholesterol using suitable methods like mean or median to ensure no empty data affects analysis.
- **Encode Categorical Features:**
  Convert categorical data (e.g., gender, lifestyle, etc.) into numerical format using techniques like Label Encoding or One-Hot Encoding, so machine learning models can process them.
- **Scale Numerical Features:**
  Normalize numerical columns using Min-Max Scaling or Standard Scaling to bring all values to a similar range, improving model performance and accuracy.
- **Split Dataset:**
  Divide the dataset into training and testing sets (e.g., 80% training, 20% testing) to evaluate how well the model performs on unseen data.
- **Final Outcome:**
  The dataset becomes clean, structured, and ready for machine learning models, ensuring better predictions and reliable results.

Task-4:

Clean and preprocess an employee salary dataset using AI scripts.

Prompt:

/content/dirty_v3_path.csvGenerate Python code to clean and preprocess a healthcare dataset. Steps: - Fill missing values in blood_pressure and cholesterol - Encode categorical variables - Scale numerical features - Split data into train and test sets

Code:

```python
import pandas as pd
import zipfile
import os
import tempfile
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Define the path to the zip file
zip_file_path = '/content/archive (2).zip'

# Create a temporary directory to extract the contents
with tempfile.TemporaryDirectory() as tmpdir:
    print(f"Extracting '{zip_file_path}' to '{tmpdir}'...")
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
        zip_ref.extractall(tmpdir)

    # List contents of the extracted directory to find the CSV file
    extracted_files = os.listdir(tmpdir)
    print(f"Contents of extracted directory: {extracted_files}")

    csv_file = None
    for f in extracted_files:
        if f.endswith('.csv'):
            csv_file = os.path.join(tmpdir, f)
            break

    if csv_file:
        print(f"Loading data from '{csv_file}'...")
        df = pd.read_csv(csv_file)
        print("Dataset loaded successfully. Displaying the first 5 rows:")
        display(df.head())
        print("\nDataset Info:")
        df.info()
        print("\nMissing values before preprocessing:")
        print(df.isnull().sum())
    else:
        print("Error: No CSV file found in the extracted archive.")
        df = pd.DataFrame() # Create an empty DataFrame to prevent further errors
```

```python
if not df.empty:
    if 'Industry' in df.columns:
        df['Industry'] = df['Industry'].str.lower()
        print("\n'Industry' column standardized to lowercase.")

    numerical_cols = df.select_dtypes(include=['number']).columns.tolist()
    categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

    for col in numerical_cols:
        if df[col].isnull().any():
            df[col] = df[col].fillna(df[col].mean())
            print(f"Missing values in '{col}' filled with mean.")

    for col in categorical_cols:
        if df[col].isnull().any():
            df[col] = df[col].fillna(df[col].mode()[0])
            print(f"Missing values in '{col}' filled with mode.")

    print("\nMissing values after initial imputation:")
    print(df.isnull().sum())

    if 'Salary(INR)' in df.columns:
        Q1 = df['Salary(INR)'].quantile(0.25)
        Q3 = df['Salary(INR)'].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df_filtered = df[(df['Salary(INR)'] >= lower_bound) & (df['Salary(INR)'] <= upper_bound)]
        num_outliers = len(df) - len(df_filtered)
        print(f"\nRemoved {num_outliers} salary outliers (outside {lower_bound:.2f}-{upper_bound:.2f}).")
        df = df_filtered.copy()
        print(f"New DataFrame shape after outlier removal: {df.shape}")
    else:
        print("\n'Salary(INR)' column not found for outlier removal.")

    X = df.copy()

    columns_to_encode = ['Location', 'Industry']
    numerical_features_for_scaling = X.select_dtypes(include=['number']).columns.tolist()

    numerical_features_for_scaling = [col for col in numerical_features_for_scaling if col not in columns_to_encode]

    numerical_transformer = StandardScaler()
```

Output:

```
Extracting '/content/archive (2).zip' to '/tmp/tmp9lc2t_q7'...
Contents of extracted directory: ['synthetic_salary_dataset.csv']
Loading data from '/tmp/tmp9lc2t_q7/synthetic_salary_dataset.csv'...
Dataset loaded successfully. Displaying the first 5 rows:
```

	Position	YearsExperience	EducationLevel	Industry	Location	Salary(INR)
0	NLP Engineer	2.8	PhD	Finance	Mumbai	1288000.0
1	Machine Learning Engineer	12.3	Bachelors	Finance	Bangalore	1488000.0
2	Computer Vision Engineer	4.8	Bachelors	Manufacturing	Remote	897000.0
3	Machine Learning Engineer	1.7	Bachelors	IT	Delhi	676000.0
4	MLOps Engineer	12.4	Masters	Healthcare	Chennai	2416000.0

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 6 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Position         500 non-null    object
 1   YearsExperience  500 non-null    float64
 2   EducationLevel   500 non-null    object
 3   Industry         500 non-null    object
 4   Location         500 non-null    object
 5   Salary(INR)      500 non-null    float64
dtypes: float64(2), object(4)
memory usage: 23.6+ KB

Missing values before preprocessing:
Position           0
YearsExperience    0
EducationLevel     0
Industry           0
Location           0
Salary(INR)        0
dtype: int64
```

```
'Industry' column standardized to lowercase.
...
Missing values after initial imputation:
Position           0
YearsExperience    0
EducationLevel     0
Industry           0
Location           0
Salary(INR)        0
dtype: int64

Removed 0 salary outliers (outside -132750.00-3119250.00).
New DataFrame shape after outlier removal: (500, 6)

Preprocessing (encoding and scaling) completed.
Shape of preprocessed data: (500, 17)
First 5 rows of preprocessed data (NumPy array):
[[-1.2529365378005226 -0.4326861593117627 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0
  1.0 0.0 0.0 0.0 0.0 'NLP Engineer' 'PhD']
 [1.025129894564064 -0.06601229567120889 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
  1.0 0.0 0.0 0.0 0.0 'Machine Learning Engineer' 'Bachelors']
 [-0.7733436046711359 -1.1495335627290453 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0
  0.0 0.0 0.0 1.0 0.0 'Computer Vision Engineer' 'Bachelors']
 [-1.516712651021685 -1.5547081820518571 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0
  0.0 0.0 1.0 0.0 0.0 'Machine Learning Engineer' 'Bachelors']
 [1.0491095412205331 1.6353544316209605 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0
  0.0 1.0 0.0 0.0 0.0 'MLOps Engineer' 'Masters']]
```

Explanation:

- **Accessing the Dataset:**
  Open the notebook and locate the dataset used (Input Data section), then navigate to the actual dataset page.
- **Downloading the Dataset:**
  Download the dataset manually as a CSV file from Kaggle to your local system.
- **Loading the Dataset:**
  Use pandas to load the downloaded file